	AIR UNIVERSITY
	DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING
	EXPERIMENT NO 12

Lab Title: **HUFFMAN**

Student Name: **Muammad Waqas** **Reg. No:** **240756**

Objective:Huffman coding and its implementation to reduce data

. BST and and real life applications

LAB ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Ability to Conduct Experiment					
Ability to assimilate the results					
Effective use of lab equipment and follows the lab safety rules					

Total Marks: _____ Obtained Marks: _____

LAB REPORT ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Data presentation					
Experimental results					
Conclusion					

Total Marks: _____ Obtained Marks: _____

Date: _____ Signature: _____

Department of Computer Engineering

Data Structures And Algorithms

LAB REPORT : 12

Tittle : Huffman coding

Name : Muammad waqas

Roll No: 240756

Submitted To : CE . Engr Maryam

Date : 14 December 24

Objective:

- **To reduce the size of data** by compressing it without losing any information.
- **To minimize storage space** required for storing files such as text, images, or data streams.
- **To assign shorter codes to frequently occurring characters** and longer codes to less frequent characters.
- **To generate optimal prefix codes** so that no code is a prefix of another, ensuring unambiguous decoding.
- **To improve transmission efficiency** by reducing the number of bits sent over a communication channel.

=======(PROBLEM STATEMENT)=====

Assignment 3

Huffman Coding is a popular lossless data compression technique that uses **binary trees** to generate variable-length prefix codes.
In this assignment, students will design and implement a complete **Huffman Compression and Decompression System** using **tree structures**.

Problem Statement

Given any text input, implement a complete system that:

A. Compresses the input

- Computes the frequency of each character.
- Builds a **Huffman Tree**.
- Generates **Huffman codes** for each character.
- Encodes the input text into a binary string (or bitstream).
- Saves the compressed data along with required metadata.

B. Decompresses the encoded data

- Reconstructs the Huffman Tree or reads metadata.
- Decodes the bitstream back into the original text.
- Verifies accuracy between original and decompressed output.

Required Tasks

Task 1 — Frequency Table

- Read input text or file.
- Count occurrences of each character.
- Display the frequency table.

Task 2 — Build Huffman Tree

- Each node stores:
    - character
    - frequency

    - left child
    - right child
- Show an intermediate representation (e.g., print tree nodes level-wise).

Task 3 — Generate Huffman Codes

- Use recursive tree traversal.
- Assign ‘0’ to left edges and ‘1’ to right edges.
- Print code table for each character.

Task 4 — Compression

- Encode the original text using Huffman codes.
- Show:
    - original size
    - compressed size
    - compression ratio
- Store the encoded bitstring in a file *(optional)*.

Task 5 — Decompression

- Read encoded data.
- Regenerate Huffman Tree or reuse stored tree metadata.
- Decode bitstream to retrieve original text.
- Validate correctness:

original_text == decompressed_text → True

=====(CODE)======

```cpp
#include <iostream> // for input/output streams
#include <string> // for string handling
#include <queue> // for priority_queue
#include <deque> // replaced vector with deque
#include <cstring> // for memset
using namespace std; // use standard namespace
// Class representing a node in Huffman tree
class HuffmanNode {
public:
unsigned char character; // character stored (only for leaf)
int frequency; // frequency of character
HuffmanNode* left; // left child
HuffmanNode* right; // right child
HuffmanNode(unsigned char ch, int freq) { // constructor to initialize node
character = ch; // assign character
frequency = freq; // assign frequency
left = right = NULL; // initialize children as null
}
};
// Comparator for priority queue to make min-heap based on frequency
class CompareNodes {
public:
bool operator()(HuffmanNode* a, HuffmanNode* b) { // comparison function
return a->frequency > b->frequency; // smaller frequency = higher priority
}
};
// Class for Huffman Encoding and Decoding
class HuffmanCoder {
```

```
29  private:
30  HuffmanNode* root; // root of Huffman Tree
31  string codes[256]; // Huffman codes for each character
32  // Recursive function to generate codes from Huffman Tree
33  void generateCodes(HuffmanNode* node, string currentCode) {
34  if (node == NULL) return; // if node is null, return
35  if (!node->left && !node->right) { // if leaf node
36  codes[node->character] = currentCode.empty() ? "0" : currentCode; // assign code
37  return;
38  }
39  generateCodes(node->left, currentCode + "0"); // go left with '0'
40  generateCodes(node->right, currentCode + "1"); // go right with '1'
41  }
42  public:
43  HuffmanCoder() { // constructor to initialize HuffmanCoder
44  root = NULL; // root initially null
45  for (int i = 0; i < 256; i++) codes[i] = ""; // initialize codes array
46  }
47  // Build Huffman Tree from frequency table
48  void buildTree(int freq[256]) {
49  priority_queue<HuffmanNode*, deque<HuffmanNode*>, CompareNodes> pq; // min-heap using deque
50  for (int i = 0; i < 256; i++) { // iterate all possible characters
51  if (freq[i] > 0) { // if character exists
52  pq.push(new HuffmanNode((unsigned char)i, freq[i])); // push node into heap
53  }}
54  if (pq.size() == 1) { // edge case: only one character
55  HuffmanNode* only = pq.top(); pq.pop(); // get the only node
56  HuffmanNode* parent = new HuffmanNode(0, only->frequency); // create parent node
```

```
57  parent->left = only; // left child points to only node
58  pq.push(parent); // push parent back to queue
59  }
60  while (pq.size() > 1) { // combine nodes until one root remains
61  HuffmanNode* a = pq.top(); pq.pop(); // get smallest freq node
62  HuffmanNode* b = pq.top(); pq.pop(); // get next smallest
63  HuffmanNode* parent = new HuffmanNode(0, a->frequency + b->frequency); // create parent
64  parent->left = a; // left child
65  parent->right = b; // right child
66  pq.push(parent); // push parent back
67  }
68  if (!pq.empty()) root = pq.top(); // final root
69  generateCodes(root, ""); // generate Huffman codes
70  }
71
72  // Encode a text string into bitstring
73  string encode(const string& text) {
74  string bitstring = ""; // initialize empty bitstring
75  for (size_t i = 0; i < text.size(); i++) { // iterate over text
76  unsigned char ch = text[i]; // get character
77  bitstring += codes[ch]; // append its code
78  }
79  return bitstring; // return encoded bitstring
80  }
81
82  // Decode bitstring into original text
83  string decode(const string& bitstring) {
84  string result = ""; // initialize result string
```

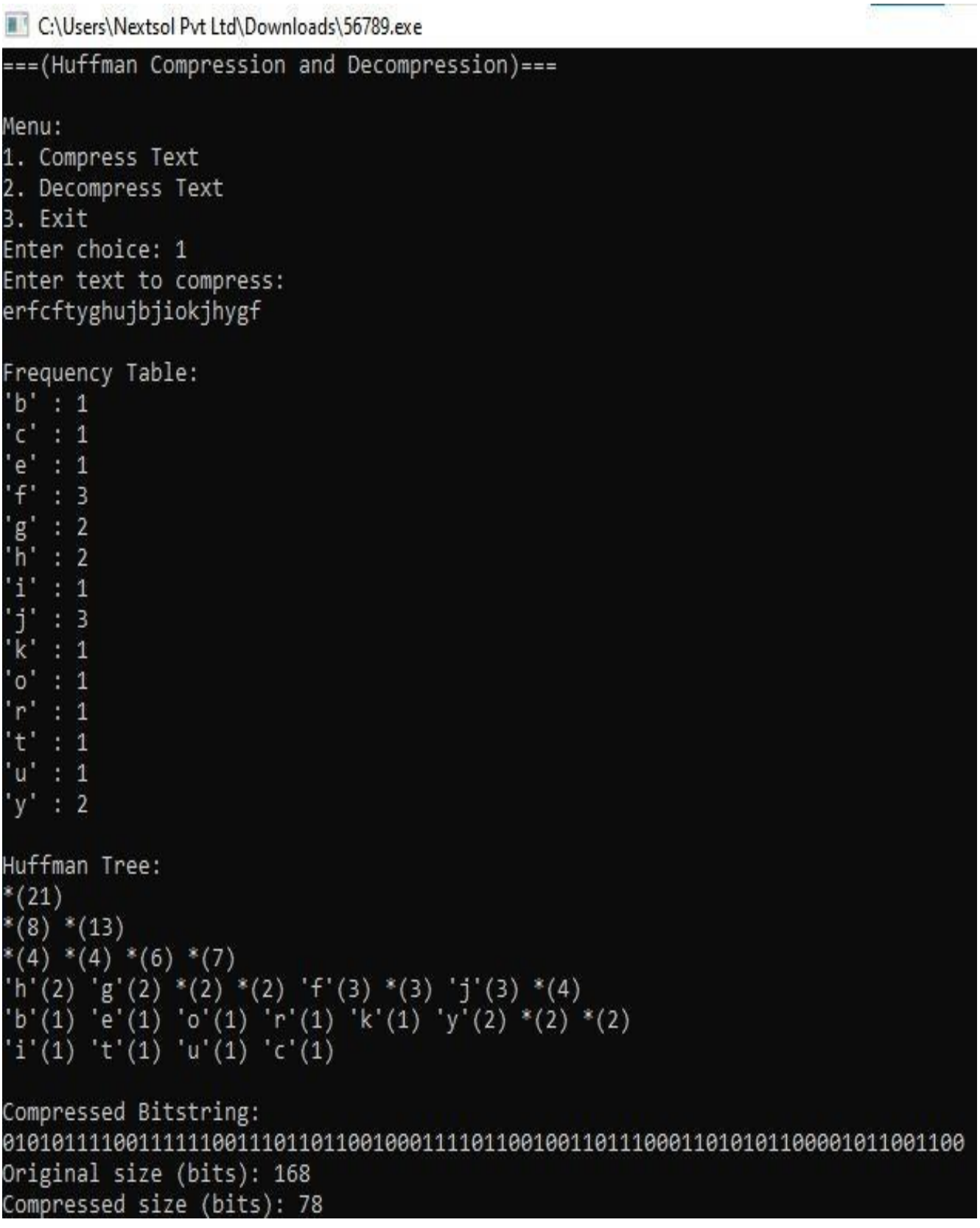
```
85  HuffmanNode* current = root; // start at root
86  for (size_t i = 0; i < bitstring.size(); i++) { // iterate bitstring
87  if (bitstring[i] == '0') current = current->left; // move left for '0'
88  else current = current->right; // move right for '1'
89  if (!current->left && !current->right) { // if leaf node
90  result.push_back(current->character); // append character
91  current = root; // reset to root
92  }}
93  return result; // return decoded string}
94  // Print Huffman codes for all characters
95  void printCodes(int freq[256]) {
96  cout << "\nHuffman Codes:\n"; // header
97  for (int i = 0; i < 256; i++) { // iterate characters
98  if (freq[i] > 0) { // if character exists
99  if (i > 31 && i < 127) cout << "'" << (char)i << "' : " << codes[i] << "\n";
100 else cout << "[" << i << "] : " << codes[i] << "\n"; // non-printable
101 }}}
102 // Print tree level-wise for visualization
103 void printTree() {
104 if (!root) { // if tree empty
105 cout << "(empty tree)\n"; // print message
106 return;
107 }
108 queue<HuffmanNode*> q; // queue for BFS
109 q.push(root); // start from root
110 while (!q.empty()) {
111 int sz = q.size(); // number of nodes in level
112 for (int i = 0; i < sz; i++) {
```

```
113  HuffmanNode* node = q.front(); q.pop(); // get node
114  if (!node->left && !node->right) { // if leaf
115  if (node->character > 31 && node->character < 127)
116  cout << "'" << node->character << "'(" << node->frequency << ") "; // printa
117  else cout << "[" << (int)node->character << "](" << node->frequency << ") ";
118  } else cout << "*(" << node->frequency << ") "; // internal node
119  if (node->left) q.push(node->left); // enqueue left
120  if (node->right) q.push(node->right); // enqueue right}
121  cout << "\n"; // end of level}}
122  // Free memory (delete tree nodes)
123  void freeTree(HuffmanNode* node) {
124  if (!node) return; // if null, return
125  freeTree(node->left); // free left subtree
126  freeTree(node->right); // free right subtree
127  delete node; // delete current node
128  }
129  ~HuffmanCoder() { // destructor
130  freeTree(root); // free entire tree
131  }};
132  // Calculate frequency table from text
133  void calculateFrequency(const string& text, int freq[256]) {
134  memset(freq, 0, sizeof(int) * 256); // initialize frequency table
135  for (size_t i = 0; i < text.size(); i++) {
136  freq[(unsigned char)text[i]]++; // count each character
137  }}
138  // Main program with menu
139  int main() {
140  cout << "===(Hufman Compression and decompression)===\n"; // header
```

```
141 int choice; // menu choice
142 string inputText; // input string
143 int freq[256]; // frequency tabl
144 while (true) { // infinite menu Loop
145 cout << "\n\n<<<<(Menu:>>>>\n";
146 cout << "1.Compress Text\n";
147 cout << "2.Decompress Text\n";
148 cout << "3.Exit\n";
149 cout << "Enter your choice: ";
150 cin >> choice; // read choice
151 cin.ignore(); // ignore newline
152 if (choice == 1) { // compress option
153 cout << "Enter text to compress:\n";
154 getline(cin, inputText); // read text
155 calculateFrequency(inputText, freq); // calculate frequency
156 HuffmanCoder coder; // create coder object
157 coder.buildTree(freq); // build Huffman tree
158 cout << "\nFrequency Table:\n"; // display table
159 for (int i = 0; i < 256; i++) {
160 if (freq[i] > 0) {
161 if (i > 31 && i < 127)cout<<"'"<<(char)i << "' : " << freq[i] << "\n";
162 else cout << "[" << i << "] : " << freq[i] << "\n";
163 }
164 }
165 cout << "\nHuffman Tree (Level-wise):\n";
166 coder.printTree(); // display tree
167 string compressedBits = coder.encode(inputText); // encode text
168 cout << "\nCompressed Bitstring:\n" << compressedBits << "\n";
```

```
169   cout << "Original size (bits): " << inputText.size() * 8 << "\n";
170   cout << "Compressed size (bits): " << compressedBits.size() << "\n";
171   } else if (choice == 2) { // decompress option
172   cout << "Enter bitstring to decompress:\n";
173   getline(cin, inputText); // read bitstring
174   cout << "Decompression requires original Huffman tree.\n";
175   cout << "Use the previous compress option first.\n";
176   } else if (choice == 3) { // exit option
177   cout << "Exiting program.\n";
178   break; // exit loop
179   } else {
180   cout << "Invalid choice. Try again.\n"; // invalid input
181   }
182   }
183   return 0; // end program
184   }
```

=====(OUTPUT)======


C:\Users\Nextsol Pvt Ltd\Downloads\56789.exe
===(Huffman Compression and Decompression)===
Menu:
1. Compress Text
2. Decompress Text
3. Exit
Enter choice: 1
Enter text to compress:
erfcftyghujbjiokjhygf
Frequency Table:
'b' : 1
'c' : 1
'e' : 1
'f' : 3
'g' : 2
'h' : 2
'i' : 1
'j' : 3
'k' : 1
'o' : 1
'r' : 1
't' : 1
'u' : 1
'y' : 2
Huffman Tree:
*(21)
*(8) *(13)
*(4) *(4) *(6) *(7)
'h'(2) 'g'(2) *(2) *(2) 'f'(3) *(3) 'j'(3) *(4)
'b'(1) 'e'(1) 'o'(1) 'r'(1) 'k'(1) 'y'(2) *(2) *(2)
'i'(1) 't'(1) 'u'(1) 'c'(1)
Compressed Bitstring:
010101111001111110011101101100100011111011001001101110001101010110000101100110
Original size (bits): 168
Compressed size (bits): 78

Decompression

```
Menu:
1. Compress Text
2. Decompress Text
3. Exit
Enter choice: 2

Decompressed Text:
erfcftyghujbjiokjhygf

Validation: Success! Original text matches decompressed text.
```

Explanation of Code:

1. What is Huffman Coding

Huffman Coding is a lossless data compression technique. It assigns shorter binary codes to characters that appear more frequently in the text and longer codes to characters that appear less frequently. This reduces the overall size of the data without losing any information.

Example: In the word hello, the letter l appears twice so it may get a shorter code than h or e.

2. Huffman Tree and Nodes

Each character and its frequency is stored in a node called HuffmanNode. A node has:

character: the actual symbol

frequency: how many times it appears

left and right: pointers to children in the tree

The tree is built by combining nodes with the smallest frequencies first.

Simple analogy: Think of combining two least-used letters into a small branch until you form a tree for the whole text.

3. Building the Huffman Tree

The tree is built using a priority queue (min-heap) to always get the two smallest frequency nodes.

Steps:

1. Push all characters with frequency greater than 0 into the heap

2. Pop two nodes with the smallest frequencies

3. Combine them into a new parent node with frequency equal to the sum of the two

4. Push the parent node back to the heap

5. Repeat until only one root node remain---

4. Generating Huffman Codes

Traverse the tree:

Left branch: add 0

Right branch: add 1

Leaf nodes get the final codes for characters.

If only one character exists, assign it 0.

Example: Text aabbbc

* a: 0

* b: 10

* c: 11

Prefix-free coding ensures that no code is a prefix of another, which avoids confusion during decoding.

5. Encoding Text

Each character in the text is converted into its Huffman code. Codes are concatenated to form a compressed bitstring.

Example: Text aabbbc becomes 0010101011.

6. Decoding Text

Start at the root of the Huffman tree. Read the compressed bitstring bit by bit:
0: move left

1: move right

:

❖ Conclusion

- After writing and running this Huffman Compression and Decompression program, I have a better understanding of how lossless data compression works. The program shows how frequently occurring characters can be encoded with shorter binary codes while rare characters use longer codes, which helps reduce the overall size of the text. By building the Huffman tree, generating codes, and encoding the text into a bitstring, I learned how computers efficiently store and transmit data without losing any information.

- The decompression part also helped me understand the importance of prefix-free codes. By following the tree structure bit by bit, the program correctly reconstructs the original text. This teaches that good coding techniques not only compress data but also ensure it can be safely and reliably decoded. Using a priority queue and tree traversal, I learned practical methods for implementing algorithms that are both fast and memory-efficient.

- Overall, practicing this code improved my understanding of real-world applications like file compression (ZIP, JPEG, MP3) and efficient data transmission. Huffman coding is a foundational concept in computer science and gives insight into how computers optimize storage and bandwidth. Writing the code from scratch and seeing it work successfully boosted my confidence in handling data structures like trees, queues, and recursion in practical projects.